

1. Dataset Overview
2. Missing Value Analysis
3. Route Type Analysis
4. Actual Time Analysis
5. OSRM Time Analysis
6. Factor Analysis
7. Facility (Hub) Analysis
8. Corridor Analysis
9. Delay Investigation
10. Data Quality Audit
11. Business Insights
12. Graph Construction Readiness
13. Phase 1 Summary

```
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd

df = pd.read_csv(
    "/content/drive/MyDrive/Dataset/delivery_data.csv"
)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

```
print("="*60)
print("DATASET SHAPE")
print("="*60)

print(df.shape)

print("\n")

print("="*60)
print("COLUMN INFO")
print("="*60)

df.info()

print("\n")

print("="*60)
print("DESCRIPTIVE STATISTICS")
print("="*60)

display(df.describe(include='all'))
```

```
=====
DATASET SHAPE
=====
(144867, 24)
```

```
=====
COLUMN INFO
=====
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0    data                                144867 non-null  object
1    trip_creation_time                 144867 non-null  object
2    route_schedule_uuid               144867 non-null  object
3    route_type                        144867 non-null  object
4    trip_uuid                         144867 non-null  object
5    source_center                     144867 non-null  object
6    source_name                       144574 non-null  object
7    destination_center                144867 non-null  object
8    destination_name                  144606 non-null  object
9    od_start_time                    144867 non-null  object
10   od_end_time                      144867 non-null  object
11   start_scan_to_end_scan            144867 non-null  float64
12   is_cutoff                        144867 non-null  bool
13   cutoff_factor                    144867 non-null  int64
14   cutoff_timestamp                  144867 non-null  object
15   actual_distance_to_destination    144867 non-null  float64
16   actual_time                      144867 non-null  float64
17   osrm_time                        144867 non-null  float64
18   osrm_distance                    144867 non-null  float64
19   factor                          144867 non-null  float64
20   segment_actual_time              144867 non-null  float64
21   segment_osrm_time                144867 non-null  float64
22   segment_osrm_distance            144867 non-null  float64
23   segment_factor                   144867 non-null  float64
dtypes: bool(1), float64(10), int64(1), object(12)
memory usage: 25.6+ MB
```

```
=====
DESCRIPTIVE STATISTICS
=====
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name
count	144867	144867	144867	144867	144867	144867	144574
unique	2	14817	1504	2	14817	1508	14
top	training	2018-09-22 04:55:04.835022	thanos::sroute:4029a8a2-6c74-4b7e-a6d8-f9e069f...	FTL	trip-153759210483476123	IND000000ACB	Gurgaon_Bilaspur_t (Haryar
freq	104858	101	1812	99660	101	23347	233
mean	NaN	NaN	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN	NaN	NaN

11 rows x 24 columns

```
missing = pd.DataFrame({
    'Missing Count': df.isnull().sum(),
    'Missing %': round(df.isnull().mean()*100,2)
})

missing = missing.sort_values(
    by='Missing Count',
    ascending=False
)

missing
```

	Missing Count	Missing %	
source_name	293	0.20	
destination_name	261	0.18	
route_schedule_uuid	0	0.00	
data	0	0.00	
route_type	0	0.00	
trip_uuid	0	0.00	
source_center	0	0.00	
trip_creation_time	0	0.00	
destination_center	0	0.00	
od_start_time	0	0.00	
od_end_time	0	0.00	
start_scan_to_end_scan	0	0.00	
is_cutoff	0	0.00	
cutoff_factor	0	0.00	
cutoff_timestamp	0	0.00	
actual_distance_to_destination	0	0.00	
actual_time	0	0.00	
osrm_time	0	0.00	
osrm_distance	0	0.00	
factor	0	0.00	
segment_actual_time	0	0.00	
segment_osrm_time	0	0.00	
segment_osrm_distance	0	0.00	
segment_factor	0	0.00	

Next steps:

[Generate code with missing](#)

[New interactive sheet](#)

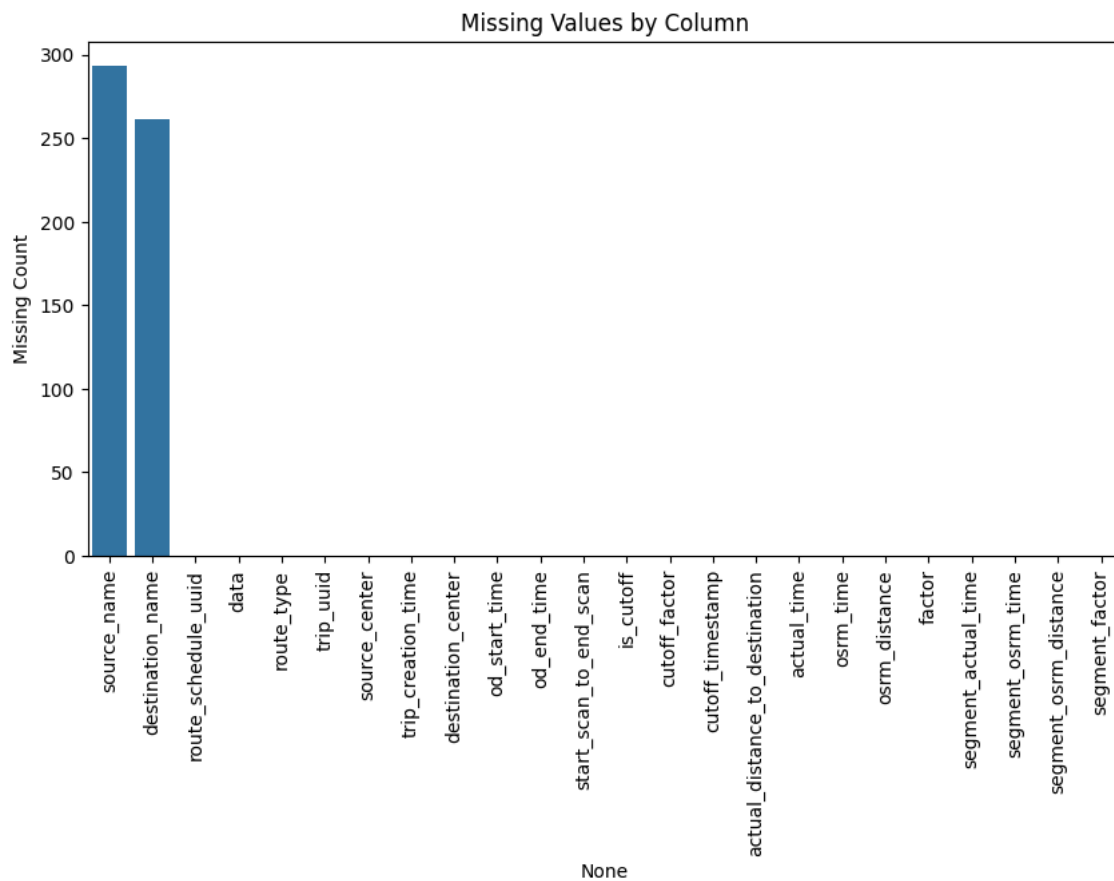
```
plt.figure(figsize=(10,5))

sns.barplot(
    x=missing.index,
    y=missing['Missing Count']
)

plt.xticks(rotation=90)

plt.title("Missing Values by Column")

plt.show()
```



```
route_counts = df['route_type'].value_counts()
```

```
route_counts
```

	count
FTL	99660
Carting	45207

```
dtype: int64
```

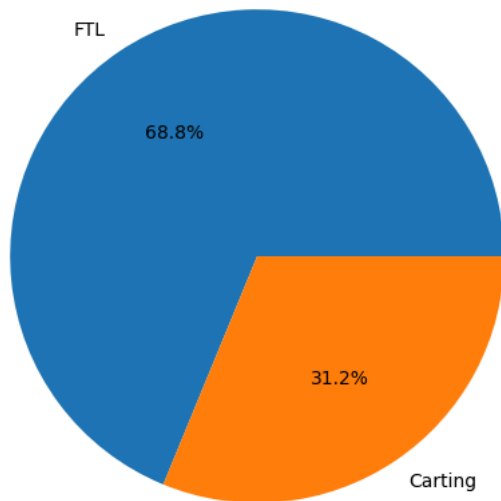
```
plt.figure(figsize=(6,6))

plt.pie(
    route_counts,
    labels=route_counts.index,
    autopct='%1.1f%%'
)

plt.title("Route Type Distribution")

plt.show()
```

Route Type Distribution



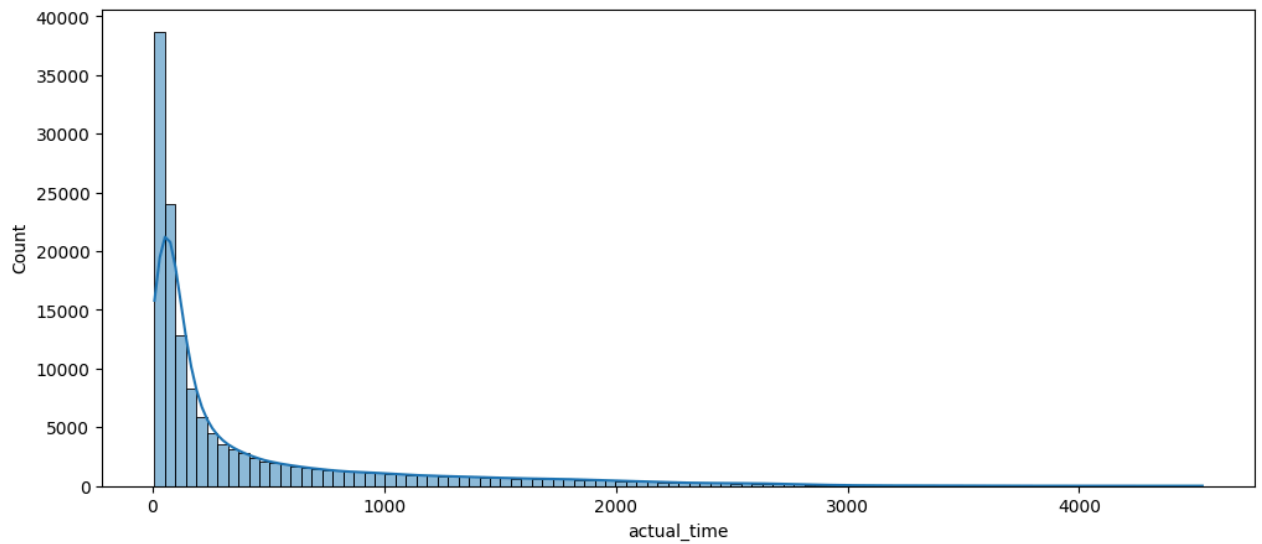
```
plt.figure(figsize=(12,5))

sns.histplot(
    df['actual_time'],
    bins=100,
    kde=True
)

plt.title("Actual Time Distribution")

plt.show()
```

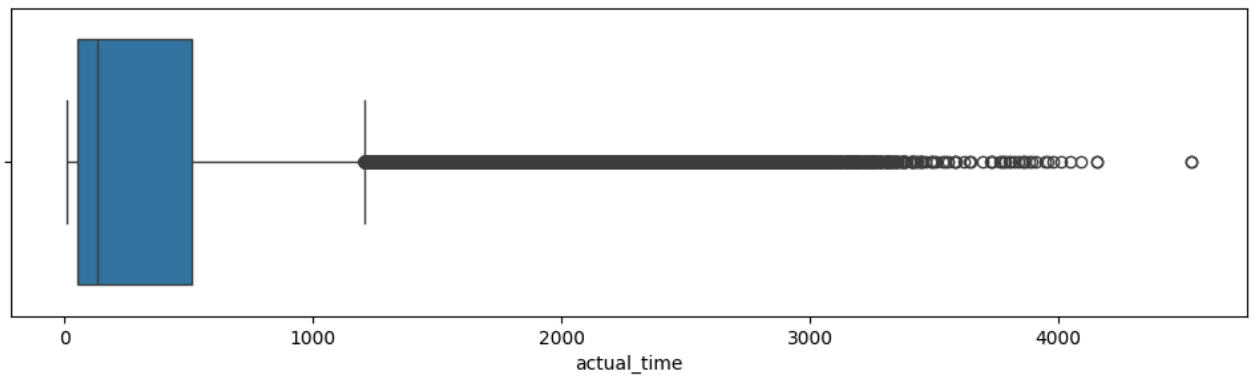
Actual Time Distribution



```
plt.figure(figsize=(12,3))

sns.boxplot(
    x=df['actual_time']
)

plt.show()
```



```
df['actual_time'].describe(
    percentiles=[0.25,0.5,0.75,0.90,0.95,0.99]
)
```

	actual_time
count	144867.000000
mean	416.927527
std	598.103621
min	9.000000
25%	51.000000
50%	132.000000
75%	513.000000
90%	1325.000000
95%	1823.000000
99%	2599.000000
max	4532.000000

dtype: float64

Business Interpretation A small number of routes likely create a disproportionate share of delays.

Potential bottleneck routes exist.

```
plt.figure(figsize=(12,5))

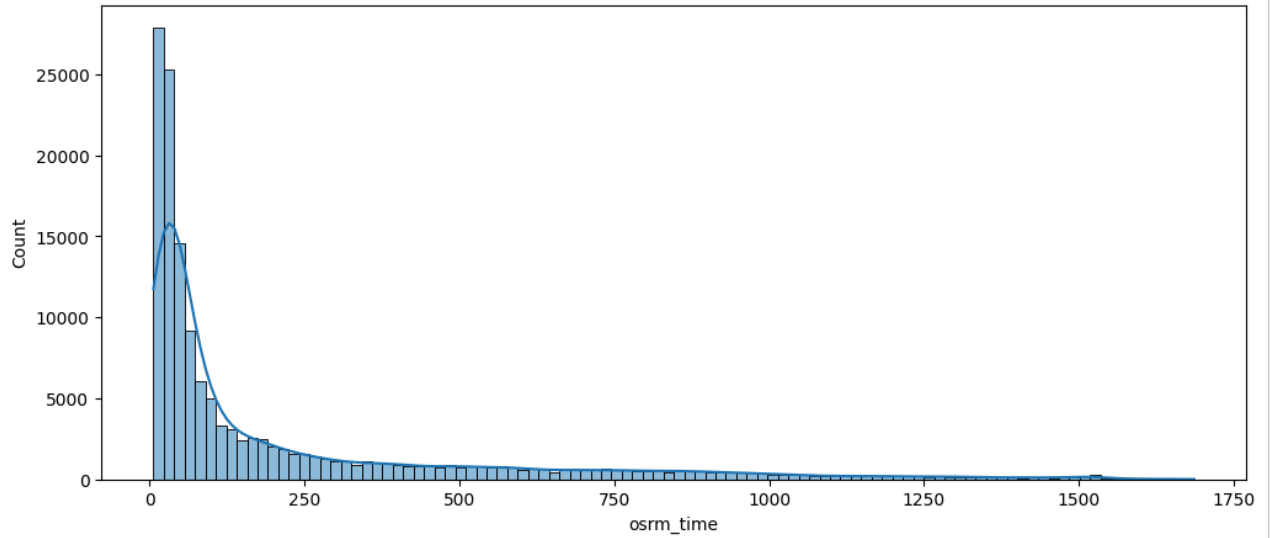
sns.histplot(
    df['osrm_time'],
    bins=100,
    kde=True
)

plt.title("OSRM Time Distribution")

plt.show()

df['osrm_time'].describe(
    percentiles=[0.25,0.5,0.75,0.90,0.95,0.99]
)
```

OSRM Time Distribution



	osrm_time
count	144867.000000
mean	213.868272
std	308.011085
min	6.000000
25%	27.000000
50%	64.000000
75%	257.000000
90%	692.000000
95%	925.000000
99%	1355.000000
max	1686.000000

dtype: float64

```
sample = df.sample(
    min(15000, len(df)),
    random_state=42
)

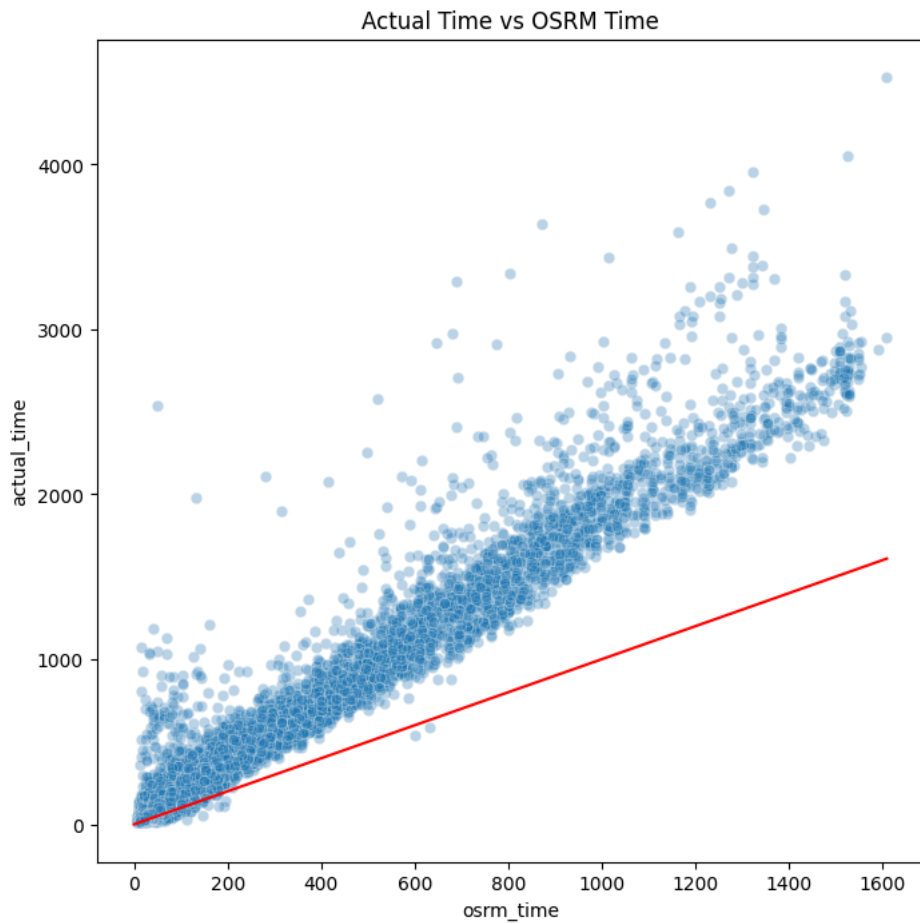
plt.figure(figsize=(8,8))

sns.scatterplot(
    data=sample,
    x='osrm_time',
    y='actual_time',
    alpha=0.3
)

plt.plot(
    [0, sample['osrm_time'].max()],
    [0, sample['osrm_time'].max()],
    color='red'
)

plt.title(
    "Actual Time vs OSRM Time"
)

plt.show()
```



Above diagonal:

Actual > OSRM

Operational delay exists.

Below diagonal:

OSRM overestimated.

```
plt.figure(figsize=(12,5))

sns.histplot(
    df['factor'],
    bins=100,
    kde=True
)

plt.title("Factor Distribution")

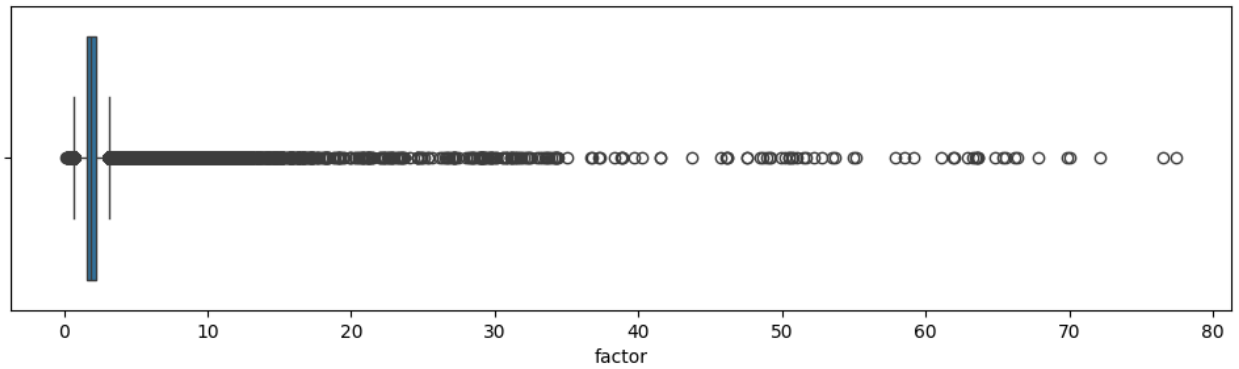
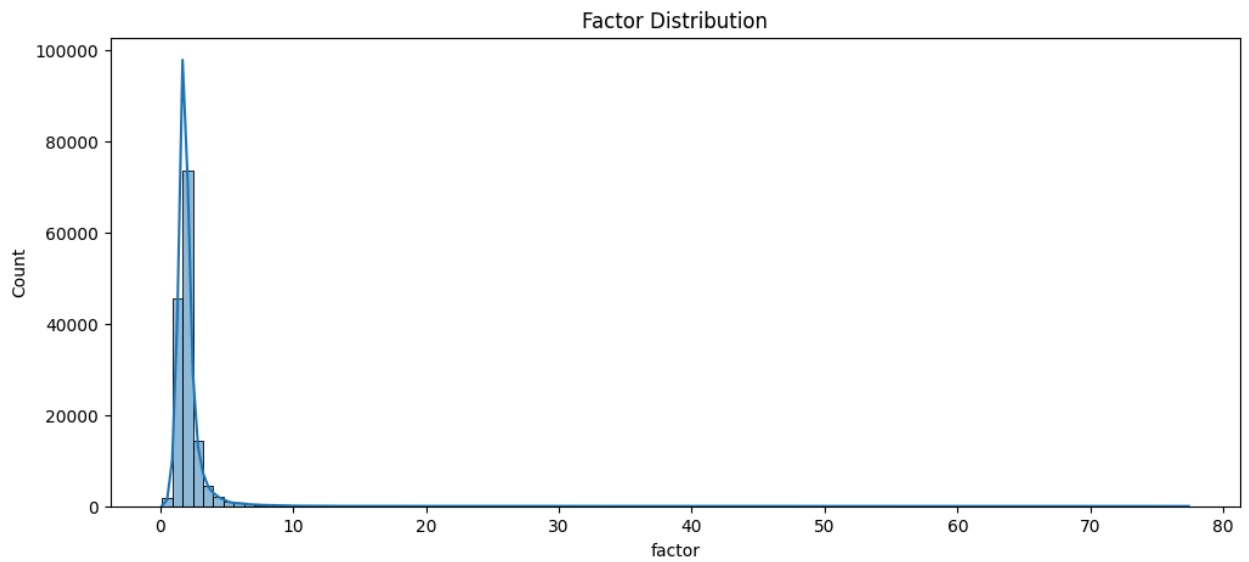
plt.show()

plt.figure(figsize=(12,3))

sns.boxplot(
    x=df['factor']
)

plt.show()

df['factor'].describe(
    percentiles=[0.5,0.75,0.90,0.95,0.99]
)
```

	factor
count	144867.000000
mean	2.120107
std	1.715421
min	0.144000
50%	1.857143
75%	2.213483
90%	2.857143
95%	3.612706
99%	6.958900
max	77.387097

dtype: float64

```
calculated_factor = (  
    df['actual_time']  
    /  
    df['osrm_time']  
)  
  
error = (  
    df['factor']  
    -  
    calculated_factor  
) .abs()  
  
error.describe()
```

```
0
count  1.448670e+05
mean    5.586886e-17
std      1.526874e-16
min      0.000000e+00
25%      0.000000e+00
50%      0.000000e+00
75%      0.000000e+00
max      7.105427e-15
```

dtype: float64

```
print((error < 0.001).mean())
```

1.0

```
all_facilities = pd.concat([
    df['source_center'],
    df['destination_center']
])

print(
    all_facilities.nunique()
)
```

1657

```
# Combine all facilities appearing as source or destination
```

```
all_facilities = pd.concat([
    df['source_center'],
    df['destination_center']
])

facility_freq = all_facilities.value_counts()

print("Unique Facilities:", all_facilities.nunique())
```

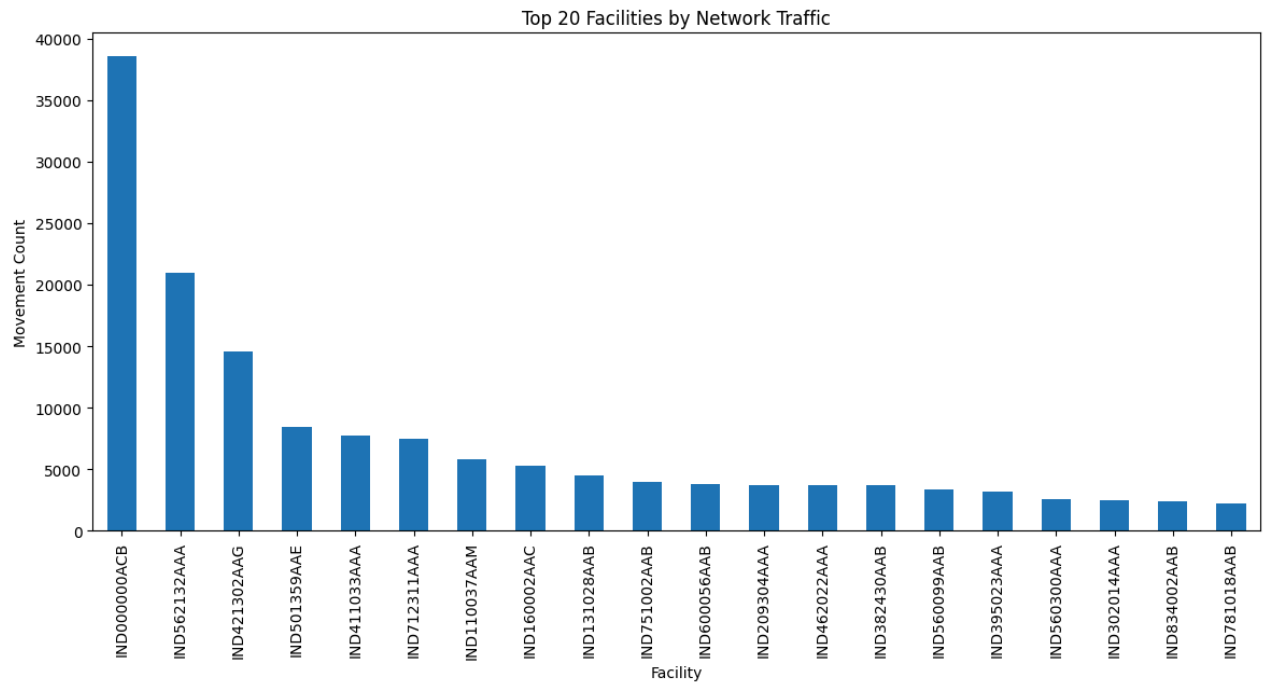
Unique Facilities: 1657

```
plt.figure(figsize=(14,6))

facility_freq.head(20).plot(
    kind='bar'
)

plt.title("Top 20 Facilities by Network Traffic")
plt.ylabel("Movement Count")
plt.xlabel("Facility")

plt.show()
```



```
df['corridor'] = (  
    df['source_center'].astype(str)  
    +  
    " -> "  
    +  
    df['destination_center'].astype(str)  
)
```

```
df['corridor'].nunique()
```

```
2783
```

```
corridor_counts = (  
    df['corridor']  
    .value_counts()  
)  
  
corridor_counts.head(20)
```

	count
corridor	
IND000000ACB -> IND562132AAA	4976
IND562132AAA -> IND000000ACB	3316
IND000000ACB -> IND712311AAA	2862
IND000000ACB -> IND501359AAE	1639
IND000000ACB -> IND421302AAG	1617
IND421302AAG -> IND000000ACB	1269
IND781018AAB -> IND110037AAM	1137
IND421302AAG -> IND562132AAA	1131
IND000000ACB -> IND411033AAA	1120
IND000000ACB -> IND600056AAB	1015
IND562132AAA -> IND302014AAA	978
IND421302AAG -> IND110037AAM	965
IND395023AAA -> IND562132AAA	935
IND411033AAA -> IND000000ACB	924
IND854326AAB -> IND000000ACB	899
IND000000ACB -> IND751002AAB	853
IND562132AAA -> IND751002AAB	818
IND110037AAM -> IND421302AAG	815
IND395023AAA -> IND110037AAM	773
IND131028AAB -> IND000000ACB	760

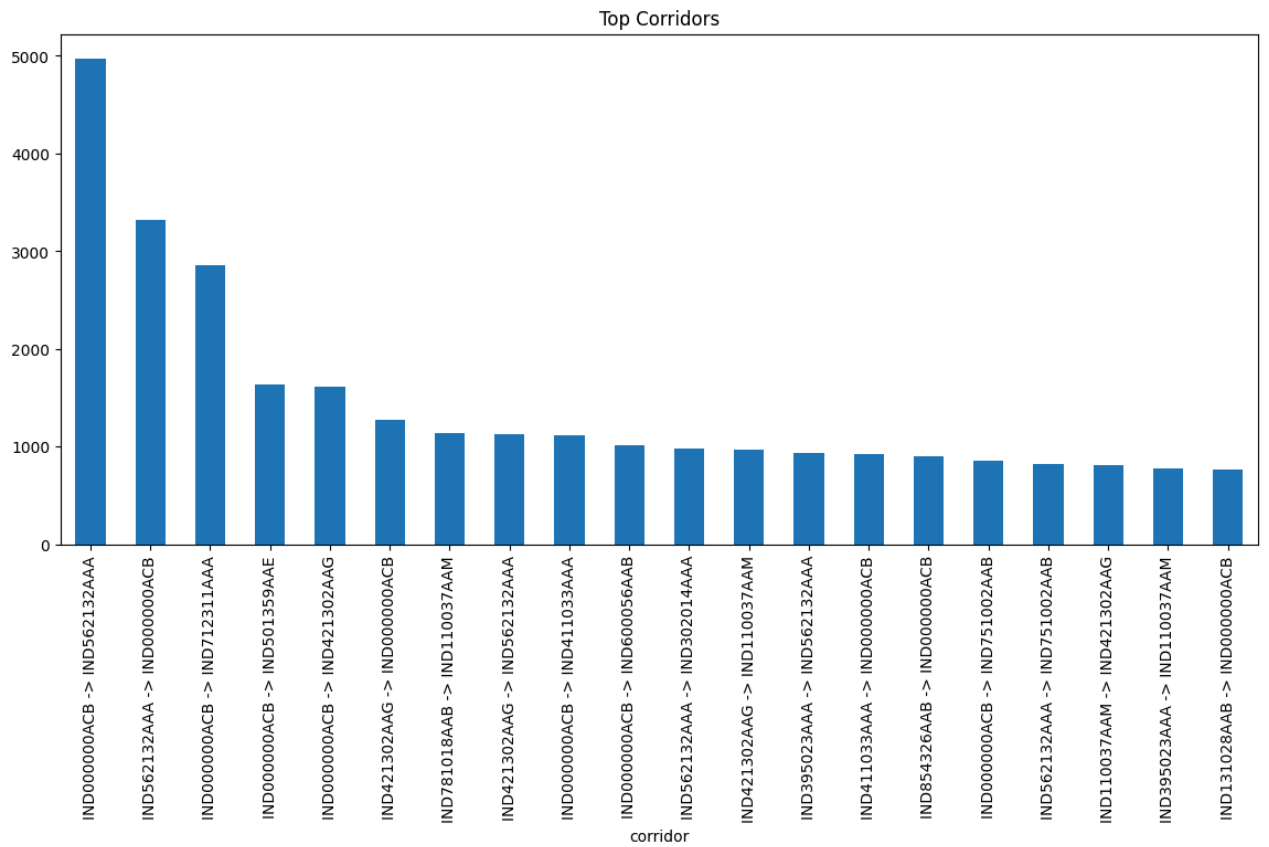
dtype: int64

```
plt.figure(figsize=(14,6))

corridor_counts.head(20).plot(
    kind='bar'
)

plt.title(
    "Top Corridors"
)

plt.show()
```



```
corridor_delay = (  
    df.groupby('corridor')['factor']  
      .mean()  
      .sort_values(  
          ascending=False  
      )  
)  
  
corridor_delay.head(20)
```

	factor
corridor	
IND212402AAA -> IND211002AAB	34.851053
IND785634AAA -> IND785001AAA	31.108333
IND842003AAB -> IND482002AAA	28.531250
IND416606AAA -> IND416510AAA	23.556107
IND395006AAA -> IND395023AAD	23.502083
IND424304AAC -> IND424006AAA	22.313663
IND208012AAA -> IND209304AAA	22.183624
IND211008AAA -> IND211002AAB	20.189616
IND722140AAA -> IND723130AAA	19.130435
IND121002AAA -> IND121004AAB	19.083333
IND283122AAB -> IND282005AAA	18.437028
IND221313AAA -> IND221401AAA	16.714286
IND425405AAA -> IND424006AAA	15.748963
IND151103AAA -> IND151001AAA	15.672365
IND743270AAA -> IND712311AAA	15.364719
IND741201AAC -> IND712311AAA	15.179825
IND785112AAC -> IND785001AAA	15.085526
IND482002AAA -> IND842003AAB	14.772321
IND493445AAB -> IND490023AAA	14.409836
IND425412AAA -> IND424006AAA	13.835443

dtype: float64

```
source_delay = (
    df.groupby('source_center')['factor']
      .mean()
      .sort_values(
          ascending=False
      )
)

source_delay.head(20)
```

	factor
source_center	
IND212402AAA	34.851053
IND785634AAA	31.108333
IND416606AAA	23.556107
IND395006AAA	23.502083
IND208012AAA	22.183624
IND211008AAA	20.189616
IND121002AAA	19.083333
IND151103AAA	15.672365
IND743270AAA	15.364719
IND741201AAC	15.179825
IND482002AAA	14.772321
IND493445AAB	14.409836
IND425405AAA	13.850094
IND431131AAB	12.441279
IND208017AAA	12.099148
IND209801AAA	11.189757
IND844505AAB	11.108328
IND792121AAB	11.097376
IND802212AAA	11.022879
IND722151AAA	10.902027

dtype: float64

```
destination_delay = (  
    df.groupby('destination_center')['factor']  
        .mean()  
        .sort_values(  
            ascending=False  
        )  
)  
  
destination_delay.head(20)
```

	factor
destination_center	
IND482002AAA	28.531250
IND416510AAA	18.641965
IND282005AAA	18.437028
IND221401AAA	16.714286
IND490023AAA	14.409836
IND232101AAB	10.510856
IND211002AAB	9.720381
IND000000AFS	9.416667
IND400037AAA	9.000000
IND233001AAA	8.893606
IND754103AAA	8.251122
IND222136AAA	7.901389
IND401501AAA	7.653921
IND425201AAA	7.342719
IND424006AAA	7.310118
IND395023AAD	7.158859
IND786181AAC	6.445322
IND411048AAA	6.320084
IND501218AAC	6.304327
IND242307AAB	6.230769

dtype: float64

```
df['segment_factor'].describe()
```

	segment_factor
count	144867.000000
mean	2.218368
std	4.847530
min	-23.444444
25%	1.347826
50%	1.684211
75%	2.250000
max	574.250000

dtype: float64

```
df[
    df['segment_factor'] < 0
].shape
```

(2365, 25)

```
df[
    df['segment_factor'] < 0
][[
    'segment_actual_time',
    'segment_osrm_time',
    'segment_factor'
]].head()
```


	segment_actual_time	segment_osrm_time	segment_factor	
47	2.0	0.0	-1.0	
54	0.0	0.0	-1.0	
90	2.0	0.0	-1.0	
164	0.0	0.0	-1.0	
224	0.0	0.0	-1.0	

```
df[
    df['segment_osrm_time'] < 1
].shape
```

(2347, 25)

```
df.sort_values(
    by='segment_factor',
    ascending=False
).head(20)
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name
5587	training	2018-09-22 11:28:06.474852	thanos::sroute:eb6f76d9-015d-4e25-bb5e-1469422...	Carting	trip-153761568647456368	IND271001AAA	Gonda_KotwaliH (Uttar Prade
15601	training	2018-09-24 09:42:45.961375	thanos::sroute:a78a08b2-ba1f-4d5a-837c-096fee6...	Carting	trip-153778216596113868	IND452001AAC	Jalgaon_ (Maharash
2081	training	2018-09-24 09:44:21.640187	thanos::sroute:c2773681-72f7-4483-8942-e031105...	FTL	trip-153778226163995868	IND580028AAA	Hubli_Adargchi (Karnata
35646	training	2018-09-17 09:26:39.998658	thanos::sroute:a78a08b2-ba1f-4d5a-837c-096fee6...	Carting	trip-153717640309053191	IND452001AAC	Jalgaon_ (Maharash
104982	training	2018-09-15 10:46:36.228538	thanos::sroute:eb6f76d9-015d-4e25-bb5e-1469422...	Carting	trip-153700839622813110	IND271001AAA	Gonda_KotwaliH (Uttar Prade
78512	training	2018-09-19 02:48:27.012436	thanos::sroute:d5ad71ae-d763-4cdb-af53-1c78952...	Carting	trip-153732530701217634	IND283122AAB	Kirauli_AchneraRC (Uttar Prade
29375	test	2018-09-27 23:45:14.576688	thanos::sroute:02de57bd-bcf5-4c26-a02e-38f1a60...	Carting	trip-153809191457632409	IND208012AAA	Kanpur_Central_I (Uttar Prade
58911	training	2018-09-12 12:49:25.626881	thanos::sroute:873bc037-283e-46a2-8135-5128fa2...	Carting	trip-153675656562660066	IND000000ACB	Gurgaon_Bilaspur_ (Harya
130915	test	2018-09-27 15:13:23.453630	thanos::sroute:abd1259c-021b-45f0-9c57-f28f118...	Carting	trip-153806120345334308	IND560058AAD	Bengaluru_Peenya (Karnata
88330	test	2018-10-01 11:27:39.027619	thanos::sroute:911f6f84-2799-473b-8702-b5e3676...	FTL	trip-153839325902727888	IND382430AAB	Ahmedabad_East_I (Guja
138440	training	2018-09-26 07:21:22.615242	thanos::sroute:e2263654-53fa-4faa-ad8a-7956bca...	FTL	trip-153794648261497343	IND580028AAA	Hubli_Adargchi (Karnata
73887	test	2018-10-01 22:33:21.977633	thanos::sroute:2db1b5d0-e54d-4f75-8f6a-dedbc99...	FTL	trip-153843320197726819	IND802212AAA	Bikramgang_Deheirc (Bit
63718	training	2018-09-17 12:22:26.351068	thanos::sroute:7e95df0e-fbc0-477a-a987-955f5b3...	FTL	trip-153718694635079685	IND382430AAB	Ahmedabad_East_I (Guja
60396	training	2018-09-23 10:40:44.455069	thanos::sroute:7e95df0e-fbc0-477a-a987-955f5b3...	FTL	trip-153769924445481836	IND382430AAB	Ahmedabad_East_I (Guja
126685	training	2018-09-13 15:00:03.482578	thanos::sroute:b7cb9e1f-84fd-4d31-abba-090ffd8...	FTL	trip-153685080348232663	IND683511AAA	Aluva_Peedikze (Ker
98266	test	2018-09-28 13:57:56.483984	thanos::sroute:78f13c07-c20c-4dd8-8ddb-254d5fa...	Carting	trip-153814307648373328	IND401104AAB	Mumbai_MiraRoad (Maharash
127959	training	2018-09-25 03:55:54.672181	thanos::sroute:ae5acc8b-0462-426a-8acd-bc9b6bb...	Carting	trip-153784775467192472	IND416606AAA	Malvan_BilvMarç (Maharash
98323	test	2018-09-28 01:54:52.978277	thanos::sroute:02473c6f-831f-4008-be51-be7290a...	FTL	trip-153809969297802591	IND571105AAA	Hunsur_RatnaDPF (Karnata
118191	training	2018-09-14 01:02:58.643475	thanos::sroute:df8d7098-cfd9-48f3-9bf3-a3144a4...	FTL	trip-153688697864318554	IND411033AAA	Pune_Tathawde (Maharash
63425	training	2018-09-21 04:15:56.079821	thanos::sroute:ae5acc8b-0462-426a-8acd-bc9b6bb...	Carting	trip-153750335607956765	IND416606AAA	Malvan_BilvMarç (Maharash

20 rows × 25 columns

```
df.groupby('route_type')[[
    'actual_time',
    'osrm_time',
    'factor'
]].mean()
```

	actual_time	osrm_time	factor	
route_type				
Carting	70.639082	30.450483	2.409842	
FTL	574.008218	297.068834	1.988680	

```
all_facilities.nunique()
```

```
1657
```

```
df['corridor'].nunique()
```

```
2783
```

```
nodes = all_facilities.nunique()
edges = df['corridor'].nunique()

density = edges / (nodes*(nodes-1))
```

```
density
```

```
0.0010142157848856703
```

```
route_summary = (
    df.groupby('route_type')[[
        'actual_time',
        'osrm_time',
        'factor'
    ]]
    .agg(['mean', 'median'])
)
```

```
route_summary
```

	actual_time		osrm_time		factor		
	mean	median	mean	median	mean	median	
route_type							
Carting	70.639082	48.0	30.450483	24.0	2.409842	1.916667	
FTL	574.008218	279.0	297.068834	145.0	1.988680	1.844679	

Next steps: [Generate code with route_summary](#) [New interactive sheet](#)

```
route_business = (
    df.groupby('route_type')[[
        'actual_time',
        'osrm_time',
        'factor'
    ]]
    .mean()
    .round(2)
)
```

```
route_business
```

	actual_time	osrm_time	factor	
route_type				
Carting	70.64	30.45	2.41	
FTL	574.01	297.07	1.99	

Next steps: [Generate code with route_business](#) [New interactive sheet](#)

```
fig, axes = plt.subplots(
    1,
    3,
    figsize=(18,5)
)

sns.barplot(
    data=df,
    x='route_type',
```